



Empresa Operativa Chile

Crear, operar y controlar una empresa chilena a través del tiempo

Contabilidad y tributación explicables · Reglas versionadas por año · Evidencia obligatoria
Sin servidor · Sin cuentas · Sin telemetría · Los datos no salen del dispositivo

Android · APK

Windows · MSI/EXE

Navegador · PWA

CLI · Node 20+

● EMPRESA REAL

Tu contabilidad, con bitácora de auditoría de cada cambio.

● SANDBOX

Empresa ficticia para practicar. Nunca toca la empresa real.

13 · Despliegue y operación

Documentación de sistema · Empresa Operativa Chile

Documento 13 de la serie

Versión del manifiesto 1.5.0 · commit 25f1a5e

Análisis del 2026-09-22

Documento técnico generado desde docs/system-documentation/. La fuente es el Markdown del repositorio: si este PDF y el Markdown difieren, manda el Markdown. Esta aplicación no presenta ni paga nada ante el SII y no es asesoría tributaria.

13 · Despliegue y operación

La conclusión, primero

Un solo `apps/web/dist` alimenta las cuatro superficies. GitHub Pages lo publica tal cual, Capacitor lo empaqueta en un APK y Tauri lo embebe en el ejecutable de Windows. No hay tres builds que puedan divergir: hay uno, y tres formas de envolverlo.

Y la regla de operación que gobierna todo lo demás: **un build en verde no prueba que el artefacto sirva**. Un APK vacío compila perfectamente. Por eso cada camino de publicación tiene un paso que

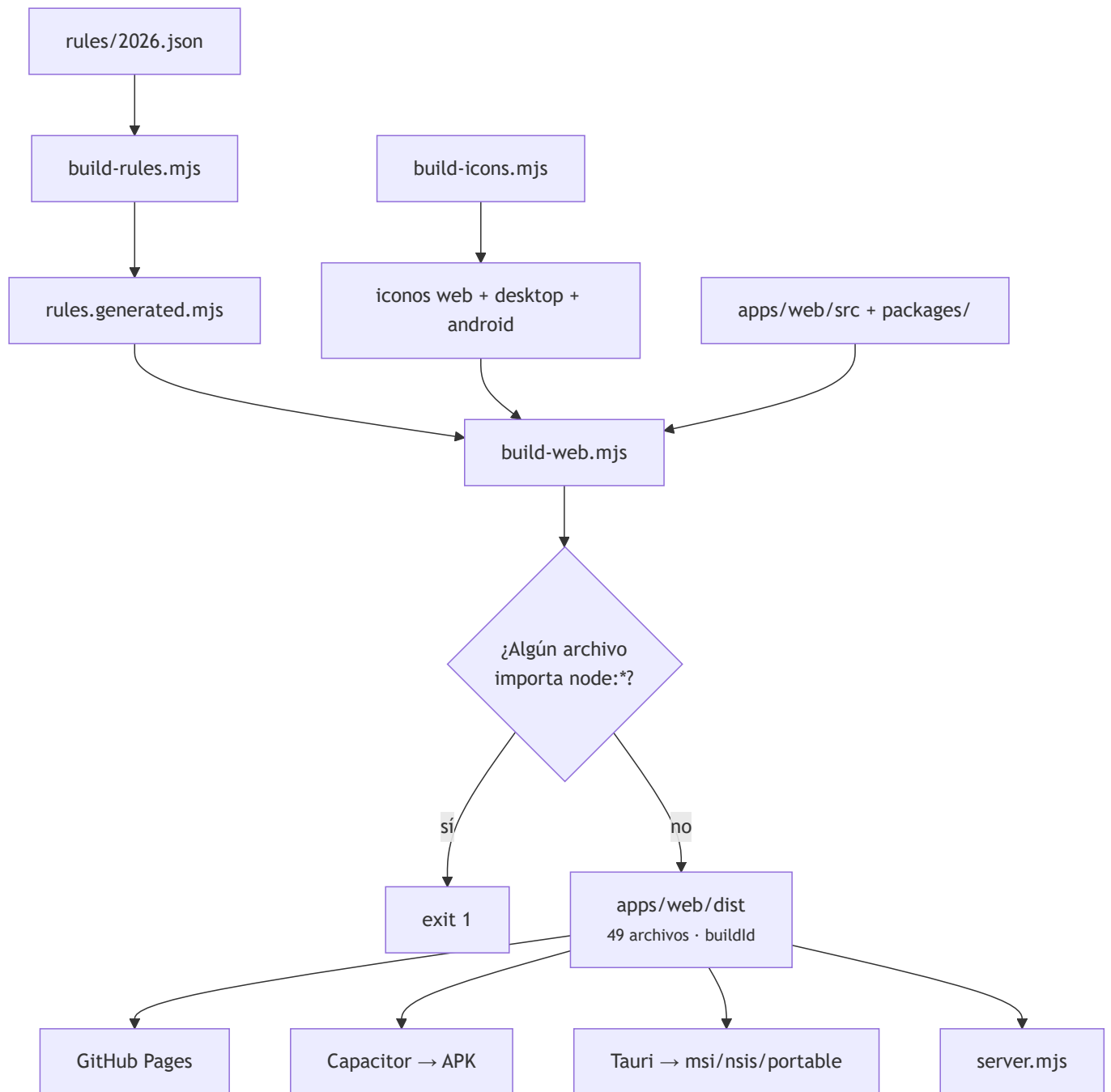
abre el resultado y cuenta lo que hay dentro antes de dejarlo salir.

Entornos

Entorno	Qué es	Cómo se llega
Local	<code>node scripts/build-all.mjs</code> + servidor en <code>127.0.0.1:4180</code>	<code>pnpm start</code>
GitHub Pages	La misma app, pública, instalable como PWA	<code>push</code> a <code>main</code> → <code>pages.yml</code>
Artefacto Android	APK de depuración, retenido 14 días	<code>android.yml</code> a mano, o dentro de un release
Artefacto Windows	<code>.msi</code> , <code>-setup.exe</code> y portable, retenidos 14 días	<code>desktop.yml</code> a mano, o dentro de un release
Release	Todo lo anterior + 3 PDF + <code>SHA256SUMS.txt</code>	Etiqueta <code>vX.Y.Z</code>

No hay staging y no hay entorno de pruebas desplegado. No hay servidor que desplegar: lo que se publica son archivos estáticos y binarios.

El proceso de construcción



El orden importa y está comentado en `build-all.mjs`: la web embebe las reglas y los iconos, así que generar la web con reglas viejas produciría un APK que calcula con la tasa del año pasado **sin que nada se ponga en rojo**.

El gate `node:*` está en el build, no en un test, y la razón está escrita al lado: si eso llega al APK, la pantalla queda en blanco sin un error visible, y un test es algo que quizá nadie corra antes de publicar. El build aborta con `exit 1` y nombra los archivos culpables.

El `buildId` es el SHA-256 de todos los archivos del `dist`, truncado a 12 caracteres. Se sustituye en `sw.js` (`__BUILD_ID__`) para invalidar el caché del service worker: publicar una versión nueva deja de dejar a la gente con una app vieja pegada en el navegador.

Salida real de esta revisión:

```
apps/web/dist listo - 49 archivos, 7256 KB, build 630ed69fe514
```

Publicación en GitHub Pages

`pages.yml` , disparado por cada push a `main` .

Paso	Qué hace
Build	<code>node scripts/build-all.mjs</code>
Verificación	Al menos 10 vistas; <code>index.html</code> contiene el nombre del producto
<code>.nojekyll</code>	Sin ese marcador, Jekyll se comería cualquier ruta que empiece por guion bajo
<code>configure-pages</code> → <code>upload-pages-artifact</code> → <code>deploy-pages</code>	Publicación

Permisos elevados sólo en este trabajo: `pages: write` e `id-token: write` . La concurrencia usa `cancel-in-progress: false` — a diferencia del resto — porque cancelar un despliegue a medias es peor que esperar.

URL: `https://vladimiracunadev-create.github.io/empresa-operativa-chile/` . **REQUIERE VALIDACIÓN :** no se consultó el estado del despliegue en esta revisión.

Recordatorio de 11 · Seguridad: **esta superficie es la única sin CSP**, porque Pages no admite cabeceras propias y el `index.html` no lleva `meta http-equiv` .

Empaquetado de Android

`android.yml` . No hay código de app propio: la app **es** la aplicación web.

Paso	Detalle
Herramientas	pnpm 11.2.2, Node 22, Temurin JDK 21
Build web	<code>node scripts/build-all.mjs</code>
Capacitor	<code>pnpm install --frozen-lockfile</code> → <code>copiar-www.mjs</code> → <code>cap add android</code> → <code>copiar-www.mjs</code> otra vez → <code>cap sync android</code>
Compilación	<code>./gradlew assembleDebug --no-daemon</code>
Verificación	<code>verify-apk.mjs</code> sobre el APK generado
Artefacto	APK + su <code>.sha256</code> , retenidos 14 días

La segunda pasada de `copiar-www.mjs` no es un error de copiar y pegar, y está comentada: `cap add` acaba de crear `android/` , y es entonces cuando se pueden aplicar los iconos propios sobre sus recursos.

`verify-apk.mjs` — 12 comprobaciones abriendo el ZIP

Lee el directorio central del ZIP **a mano**, sin `unzip` ni librerías, para correr igual en Windows, Linux y macOS.

#	Comprobación	Umbral
1–4	<code>index.html</code> , <code>app.js</code> , <code>app.css</code> y el manifiesto PWA están dentro	Presencia
5	Las vistas viajan	≥ 10
6	El núcleo de cálculo viaja	≥ 6 módulos <code>.mjs</code>
7	Los iconos viajan	≥ 3
8–10	<code>AndroidManifest.xml</code> , <code>resources.arsc</code> , <code>classes.dex</code>	Presencia
11	Tamaño razonable	> 1 MB y < 120 MB
12	Las reglas tributarias viajan dentro	<code>core/chile-tax-rules/rules.generated.mjs</code>

Si alguna falla: `este APK NO debe publicarse` y `exit 1` .

El comentario del propio script explica por qué existe, y vale la pena leerlo: no comprueba que el build funcionara, comprueba **que el resultado sirva**. Un APK vacío arranca la WebView, muestra una pantalla en blanco y deja todas las señales del build en verde.

El APK es de depuración, firmado con la clave de debug de Android. Sirve para instalar y usar, no para publicar en Google Play. Está declarado en el resumen del propio workflow y en las notas de cada release.

Empaquetado de Windows

`desktop.yml` , sobre `windows-latest` , hasta 45 minutos.

Paso	Detalle
Herramientas	pnpm 11.2.2, Node 22, Rust <code>stable</code> , caché de Rust (<code>Swatinem/rust-cache</code>)
Build web	<code>node scripts/build-all.mjs</code>
Verificación previa	≥ 10 vistas, ≥ 6 módulos del núcleo, el nombre en <code>index.html</code> , <code>f29Basic</code> en el motor
Pruebas de Rust	<code>cargo test --release</code>
Compilación	<code>pnpm install --frozen-lockfile</code> → <code>pnpm build</code>
Recogida	<code>.msi</code> , <code>-setup.exe</code> y el portable; falla si falta cualquiera
Umbral de tamaño	Falla si un artefacto pesa menos de 1 MB
<code>SHA256SUMS.txt</code>	Calculado en PowerShell
Prueba de arranque	Lanza el portable, espera 12 s y falla si el proceso murió

La verificación va antes de empaquetar, y está comentado por qué: Tauri embebe y comprime los recursos dentro del binario, así que después ya no se pueden contar leyendo el `.exe` . El momento de comprobar que la app está entera es justo antes de sellarla.

Dos detalles de PowerShell que el workflow documenta y que conviene no deshacer:

- La lista de archivos se materializa **antes** de escribir `SHA256SUMS.txt` : encadenar

`Get-ChildItem | ... | Out-File` en el mismo directorio hace que `Out-File` abra el archivo mientras la enumeración sigue viva, y `Get-FileHash` acaba leyendo el archivo que se está escribiendo.

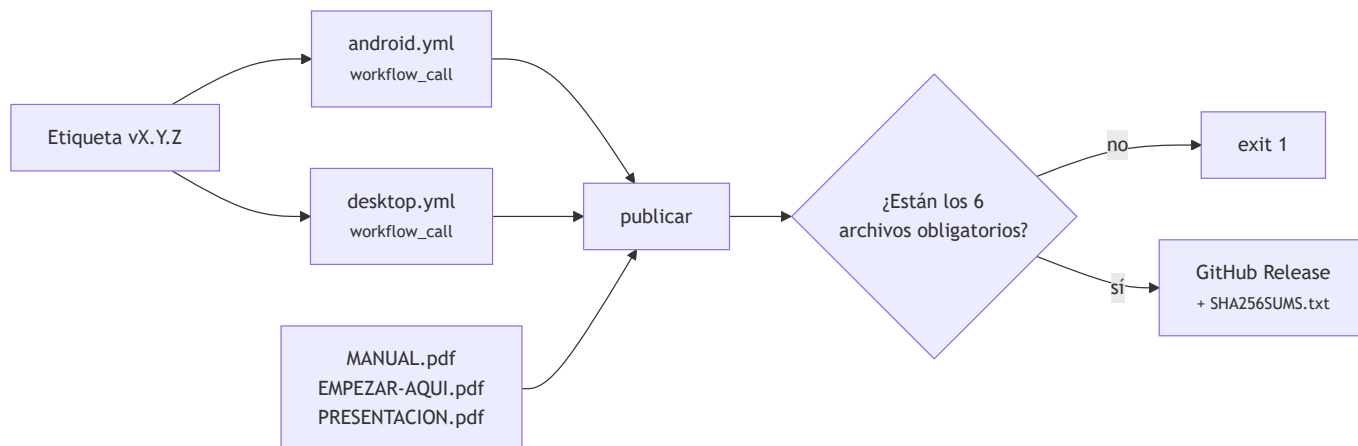
- No se le puede pedir a una app de ventana que responda por HTTP, pero sí comprobar que el proceso levanta y sigue vivo: eso descarta los fallos de arranque por DLL o recursos ausentes.

Los tres formatos existen porque resuelven situaciones distintas: `.msi` para despliegue corporativo o instalación desatendida, `-setup.exe` (NSIS) para una persona, y el portable para un pendrive o un equipo ajeno.

Los instaladores no están firmados con certificado de código: SmartScreen avisará la primera vez. Declarado en el resumen del workflow.

Release

`release.yml`, disparado por una etiqueta `v*`.



Reutiliza los workflows en vez de duplicar sus pasos: lo que se publica es exactamente lo que esos workflows verificaron.

La documentación viaja con el release —manual, guía «Empezar aquí» y presentación en PDF— porque quien descarga un instalador debería poder llevarse las instrucciones sin volver al repositorio.

Un release incompleto no se publica: hay seis `test -f` que fallan el trabajo si falta el APK, cualquiera de los tres instaladores, el manual o la guía.

Las notas se redactan en el propio workflow e incluyen, además de las descargas, una sección de

advertencias honestas: binarios sin firmar, APK de depuración, la app no presenta ni paga nada ante el SII, y el borrador F29 es una ayuda de control y no una declaración.

Publicar una versión

```
git tag v1.4.1
git push origin v1.4.1
```

Antes conviene haber actualizado `CHANGELOG.md`, la `version` de `package.json` y la de `tauri.conf.json` — nada comprueba que las dos coincidan (ver [10 · Configuración](#)).

Generación de los PDF de esta documentación

`scripts/build-system-docs.mjs`, escrito para esta serie.

```
node scripts/build-system-docs.mjs          # todos + el consolidado
node scripts/build-system-docs.mjs --only 03 # sólo el 03, para iterar rápido
node scripts/build-system-docs.mjs --no-diagrams # sin rasterizar Mermaid
```

Cómo funciona:

1. Lee cada `.md` de `docs/system-documentation/` en orden, con el `README.md` primero.
 2. Quita las barras de navegación entre documentos: son útiles en GitHub y ruido en papel.
 3. **Rasteriza cada bloque Mermaid a SVG** con `mmdc`, cacheado por hash del contenido en `assets/diagramas/`. Regenerar sin tocar un diagrama no vuelve a invocar el rasterizador, que es con diferencia el paso más lento.
1. Convierte a HTML con el mismo `scripts/lib/markdown.mjs` que usan el manual y la guía, embebiendo cada imagen como data URI.
 1. Compone una portada con **datos que no se escriben a mano**: la versión sale de `package.json`, el commit de `git rev-parse --short HEAD` y la fecha de análisis del propio `README.md`.
 1. Imprime a PDF con `scripts/lib/chrome.mjs` y la hoja `PRINT_CSS` compartida.
 2. Genera además `documentacion-completa.pdf` con la serie entera.

Tres decisiones que conviene conocer antes de tocarlo:

- **Si `mmdc` no está instalado, no falla.** Deja los diagramas como bloques de código y lo avisa por consola. Un PDF con un diagrama en texto es peor que uno con el diagrama dibujado, y mucho mejor que ningún PDF.
- **Avisa cuando un diagrama sale ilegible.** Calcula el ancho útil de una A4 (688 px con los márgenes de `PRINT_CSS`) y, si el diagrama se reduce por debajo del 45 %, lo dice con nombre y factor. Es lo que detectó que el diagrama entidad-relación de 07 medía 2.328 px y se imprimía al 30 %; por eso ese diagrama está partido en dos.
- **Pide las imágenes en carga inmediata.** `markdownToHtml` marca las imágenes como `loading="lazy"`, que es correcto para el HTML del manual y un riesgo al imprimir: un diagrama que el navegador considere fuera de vista puede no haberse decodificado al dispararse la impresión, y sale un hueco.

Requisito: Chrome o Edge (autodetectado, o `CHROME_PATH`). Para los diagramas, `@mermaid-js/mermaid-cli` global:

```
pnpm add -g @mermaid-js/mermaid-cli
```

No está integrado en `pnpm check` ni en CI, a diferencia de los generadores del glosario y la guía. **INFERENCIA**: integrarlo exigiría Chrome y `mmdc` en el runner, y ninguno de los dos está hoy en los workflows.

La consecuencia es que **los PDF de esta serie hay que regenerarlos a mano cuando cambie el Markdown;** se recoge en [15 · Riesgos](#).

Otros documentos que se generan

Comando	Qué produce
<code>pnpm docs</code>	Capturas + guía + manual + presentación
<code>pnpm manual</code>	<code>docs/MANUAL.html</code> y <code>.pdf</code>
<code>pnpm guia</code>	<code>docs/EMPEZAR-AQUI.md</code> , <code>.html</code> y <code>.pdf</code>
<code>pnpm presentacion</code>	Diapositivas y pauta, y las verifica
<code>pnpm capturas</code>	Capturas desde la app servida en <code>CAPTURE_URL</code>

Los cinco `--check` correspondientes corren en CI: si alguien edita el documento generado a mano, el build falla.

Operación

Logs, métricas, monitoreo y alertas

`NO IDENTIFICADO` , y es coherente con el diseño: **no hay servidor que observar.**

Aspecto	Estado
Logs de servidor	El servidor local imprime su URL al arrancar y nada más. No escribe archivo de log
Logs de la aplicación	<code>console.warn</code> cuando falla el espejo de disco o la hidratación. Nada más
Métricas	Ninguna. No hay telemetría, por decisión de producto
Monitoreo	Ninguno
Alertas	Las de GitHub: fallo de workflow y hallazgos de CodeQL
Trazas	Ninguna

Lo más parecido a observabilidad que tiene el producto es la bitácora de auditoría, y sirve para que el usuario entienda su propia empresa, no para que nadie observe el sistema.

Respaldo y recuperación

El respaldo es del usuario, y es la contrapartida directa de no tener servidor.

Escenario	Recuperación
Se borran los datos del navegador	Importar el último respaldo exportado. Sin respaldo, no hay recuperación
Se desinstala el APK	Igual
Se corrompe el espejo de Windows	<code>localStorage</code> sigue siendo la fuente; el espejo se reescribe en la siguiente escritura
Se corrompe <code>localStorage</code> en Windows	Al arrancar, <code>hydrateFromDisk()</code> repone lo que falte desde el espejo
Se pierde el dispositivo	Sólo el respaldo exportado fuera de él

`SECURITY.md` da la regla operativa: **exportar desde la pestaña Datos cada vez que se cierra un período**, y guardar el archivo fuera del dispositivo.

Rollback

Qué	Cómo
GitHub Pages	Revertir el commit y empujar: <code>pages.yml</code> republica
Release	Instalar el artefacto de la versión anterior desde su release
Datos	Importar un respaldo anterior. Un ejercicio ya cerrado no se pisa al importar en modo fusión
Reglas tributarias	Revertir el <code>.json</code> y regenerar, o CI falla en <code>--check</code>

No hay migración de datos que revertir, porque no hay migraciones destructivas: las dos que existen ocurren en lectura y no tocan el almacén.

Mantenimiento periódico

Cadencia	Tarea
Anual, antes de que empiece el año comercial	Crear <code>rules/<año>.json</code> , verificar cada tasa contra su fuente oficial , regenerar y publicar
Mensual	Actualizar la UTM del mes en el archivo de reglas si se van a calcular patentes
Semanal	Revisar los hallazgos de CodeQL (el workflow corre los lunes a las 06:00 UTC)
Por release	<code>CHANGELOG.md</code> , las dos <code>version</code> , regenerar documentación
Cuando cambie el Markdown de esta serie	<code>node scripts/build-system-docs.mjs</code>

La tarea anual es la que sostiene el producto. Todo lo demás es higiene; esa es la que evita que la aplicación calcule con la tasa del año pasado. `CONTRIBUTING.md` la formaliza: una tasa que se toca exige su fuente oficial y su fecha de verificación.

Lo que no pudo verificarse

Aspecto	Por qué
Compilación real del APK y del instalador	Faltan JDK/SDK/Gradle y toolchain de Rust en la máquina de análisis
<code>verify-apk.mjs</code> sobre un APK real	No hay APK que verificar
Estado del último despliegue en Pages	No se consultó el repositorio remoto
Que el artefacto publicado coincida con el verificado	<code>sha256sum</code> del archivo descargado contra el <code>SHA256SUMS.txt</code> del release
Comportamiento del instalador NSIS y del MSI	No se ejecutaron